PERFORMANCE REPORT

Assessment performance

Full Stack Developer Assessment

ID 149c8b26-3ad8-469a-822e-f1f8f0dcdc7a

STUDENT

Ahamad Ali

Email: shailkahamadali3773@gmail.com

YOUR SCORE

88.0 / 100

Strong Performance

Score attainment	**88.0%**
Accuracy	**95.7%**
Percentile	**61.4%**
Status	Submitted

This report summarizes outcomes for "**Full Stack Developer Assessment**". Overall accuracy is **95.65%**, based on attempted items and topic-level performance.

Questions attempted

23

Out of 23 total items · submitted

Top three focus areas

Function definition
Return statement
Conditional statements

Strongest skills in this attempt (when provided by the assessment).

Answer breakdown

Correct 96%

Incorrect 4%

Unattempted 0%

Score & readiness

Score / max 88%

Accuracy 96%

Placement readiness 5%

Percentile 61%

Topic accuracy

Python Fundamentals 100%

FastAPI & REST APIs 100%

ReactJS Fundamentals 100%

SQL & PostgreSQL 100%

Time usage

Time used (vs allotted) 37%

33 / 90 minutes

Skills needing attention

Focus your next study blocks on these areas - accuracy reflects this attempt only.

Sorting 0%

0/1 correct · practice priority

Date Manipulation 0%

0/1 correct · practice priority

Hash Table 0%

0/1 correct · practice priority

Feedback

- Strong accuracy on attempted questions — keep reinforcing these strengths.
- You finished with time to spare — use extra minutes to double-check tricky questions next time.

Quiz responses (20)

Q1 of 20 · Correct · Your answer: D · Easy · Python Fundamentals

Which of the following is the correct way to define a function in Python that takes two parameters and returns their sum?

A. def add(a, b): print(a + b)

B. function add(a, b) { return a + b }

C. add(a, b) => a + b

D. def add(a, b): return a + b (correct · your answer)

Explanation

In Python, a function is defined using the 'def' keyword, followed by the function name and parameters in parentheses. The 'return' statement sends back the result. Option D correctly shows this syntax and returns the sum of the two parameters.

Q2 of 20 · Correct · Your answer: D · Easy · Python Fundamentals

What will be the output of the following code? x = 5 if x > 3: print('Greater') else: print('Smaller or equal')

A. No output

B. Error due to syntax

C. Smaller or equal

D. Greater (correct · your answer)

Explanation

Since the variable x is 5, and 5 is greater than 3, the condition in the if statement evaluates to True, so 'Greater' is printed.

Q3 of 20 · Correct · Your answer: D · Medium · Python Fundamentals

What will be the output of the following Python code snippet? x = [1, 2, 3] y = x x.append(4) print(y)

A. [4]

B. [1, 2, 3]

C. Error, because y is not defined

D. [1, 2, 3, 4] (correct · your answer)

Explanation

In Python, variables that reference mutable objects like lists point to the same object in memory. Here, both x and y refer to the same list. When 4 is appended to x, y reflects this change because they reference the same list object. Therefore, y prints as [1, 2, 3, 4].

Q4 of 20 · Correct · Your answer: A · Medium · FastAPI & REST APIs

In FastAPI, which feature allows automatic generation of interactive API documentation for REST endpoints?

A. Using the OpenAPI standard with automatic schema generation (correct · your answer)

B. Integrating third-party API documentation tools only

C. Manual creation of HTML docs using Jinja2 templates

D. Writing Swagger UI code manually for each route

Explanation

FastAPI automatically generates API documentation using the OpenAPI standard by inspecting the code and type annotations, which allows tools like Swagger UI and ReDoc to render interactive docs without manual intervention.

Q5 of 20 · Correct · Your answer: B · Medium · FastAPI & REST APIs

When designing a REST API with FastAPI, which HTTP method and status code pair best represents a successful resource update?

A. DELETE method with status code 202 Accepted

B. PUT method with status code 200 OK (correct · your answer)

C. GET method with status code 204 No Content

D. POST method with status code 201 Created

Explanation

The PUT method is typically used to update an existing resource. A successful update usually returns status code 200 OK, indicating the request was successful and the resource was updated.

Q6 of 20 · Correct · Your answer: A · Easy · FastAPI & REST APIs

In FastAPI, which HTTP method is typically used to retrieve data from a REST API endpoint?

A. GET (correct · your answer)

B. PUT

C. POST

D. DELETE

Explanation

The GET method is used to request data from a specified resource without causing any changes on the server, which aligns with retrieving information in REST APIs.

Q7 of 20 · Correct · Your answer: C · Easy · FastAPI & REST APIs

What is the main advantage of using FastAPI for building REST APIs compared to traditional frameworks?

A. It requires manual validation of data

B. It only supports XML data format

C. It automatically generates interactive API documentation (correct · your answer)

D. It uses synchronous programming exclusively

Explanation

FastAPI automatically generates interactive API documentation (Swagger UI and ReDoc) based on the code, which simplifies development and testing of REST APIs.

Q8 of 20 · Correct · Your answer: D · Easy · ReactJS Fundamentals

In React, what is the primary purpose of using a component?

A. To store data permanently on the client-side

B. To handle server-side rendering exclusively

C. To directly manipulate the DOM elements for faster rendering

D. To create reusable UI elements that manage their own state and props (correct · your answer)

Explanation

React components allow developers to build encapsulated, reusable pieces of UI that can manage their own state and receive data through props, promoting modular and maintainable code.

Q9 of 20 · Correct · Your answer: C · Easy · ReactJS Fundamentals

Which of the following correctly describes the role of 'props' in React components?

A. Props are used to store component-specific state data internally

B. Props handle lifecycle events of a React component

C. Props are immutable inputs passed from parent to child components (correct · your answer)

D. Props allow components to update the DOM directly

Explanation

Props (properties) are read-only inputs passed from parent components to child components, enabling data flow and configuration without being modifiable by the receiving component.

Q10 of 20 · Correct · Your answer: C · Medium · ReactJS Fundamentals

In ReactJS, what is the primary reason for using keys when rendering lists of components?

A. To prevent the list from rendering when keys are missing

B. To ensure the order of elements is reversed during rendering

C. To uniquely identify elements for efficient updates and re-rendering (correct · your answer)

D. To apply CSS styles to list elements more easily

Explanation

Keys in React are used to uniquely identify elements in a list so that React can efficiently determine which items have changed, been added, or removed, allowing for optimal re-rendering performance.

Q11 of 20 · Correct · Your answer: A · Hard · ReactJS Fundamentals

In ReactJS, which of the following best explains the behavior and use of the 'key' prop in lists when rendering dynamic components?

A. The 'key' prop helps React identify which items have changed, are added, or removed, optimizing re-rendering by minimizing DOM updates. (correct · your answer)

B. The 'key' prop automatically binds event handlers to list items to improve performance during state changes.

C. The 'key' prop is used to pass unique identifiers to child components for accessing their internal state directly.

D. The 'key' prop enables two-way data binding between the parent and dynamically generated child components.

Explanation

The 'key' prop in React is a special attribute that helps React identify elements in a list uniquely. This identification allows React to efficiently update and reorder items by minimizing the number of DOM manipulations during re-rendering. It does not bind event handlers, enable two-way binding, or provide direct access to child state.

Q12 of 20 · Correct · Your answer: C · Easy · SQL & PostgreSQL

In PostgreSQL, which SQL command is used to add a new row of data into a table?

A. DELETE
B. UPDATE
C. INSERT INTO (correct · your answer)
D. SELECT

Explanation

The INSERT INTO command is used to add new records (rows) to a table. UPDATE modifies existing rows, SELECT retrieves data, and DELETE removes rows.

Q13 of 20 · Correct · Your answer: B · Easy · SQL & PostgreSQL

Which data type in PostgreSQL is best suited for storing whole numbers without decimals?

A. BOOLEAN
B. INTEGER (correct · your answer)
C. TEXT
D. VARCHAR

Explanation

INTEGER is the appropriate data type for storing whole numbers in PostgreSQL. VARCHAR and TEXT are for strings, BOOLEAN stores true/false values.

Q14 of 20 · Correct · Your answer: B · Medium · SQL & PostgreSQL

In PostgreSQL, which of the following SQL statements correctly retrieves all rows from the table 'employees' where the salary is above the average salary of the entire table?

A. SELECT * FROM employees WHERE salary > ALL(SELECT salary FROM employees);
B. SELECT * FROM employees WHERE salary > (SELECT AVG(salary) FROM employees); (correct · your answer)
C. SELECT * FROM employees WHERE salary > MAX(salary);
D. SELECT * FROM employees WHERE salary > AVG(salary);

Explanation

Option B correctly uses a subquery to calculate the average salary and compares each employee's salary to that average. Option A incorrectly uses ALL with salaries instead of the average. Option C compares salary to the maximum salary, which is not the average. Option D is invalid because aggregate functions cannot be used directly in the WHERE clause without a subquery.

Q15 of 20 · Correct · Your answer: A · Hard · SQL & PostgreSQL

In PostgreSQL, which of the following statements about transaction isolation levels is TRUE when using the default 'Read Committed' isolation level?

A. A query in a transaction sees only data committed before the query began, not data committed by concurrent transactions after the query starts. (correct · your answer)

B. A transaction sees a consistent snapshot of the database as of the start of the transaction, regardless of concurrent changes.

C. Non-repeatable reads and phantom reads cannot occur under 'Read Committed' isolation level.

D. Dirty reads are allowed, meaning a transaction can read uncommitted changes made by other transactions.

Explanation

'Read Committed' isolation in PostgreSQL provides statement-level snapshotting, meaning each query sees data committed before that query started. This prevents dirty reads but allows non-repeatable reads and phantom reads because subsequent queries in the same transaction may see new commits by others. Only 'Repeatable Read' and higher prevent these phenomena.

Q16 of 20 · Correct · Your answer: B · Easy · Data Structures & Problem Solving

Which data structure is most suitable for implementing a function call stack in a program?

A. Queue

B. Stack (correct · your answer)

C. Linked List

D. Array

Explanation

A stack is a Last-In-First-Out (LIFO) data structure that naturally fits the function call behavior where the most recent call must be completed before returning to the previous one.

Q17 of 20 · Correct · Your answer: D · Medium · Data Structures & Problem Solving

You are given a singly linked list and asked to detect if it contains a cycle. Which of the following approaches offers the most efficient way to detect a cycle without modifying the list?

A. Count the length of the list and if it exceeds a certain threshold, conclude a cycle exists.

B. Reverse the linked list and check if the head changes; if it does, a cycle exists.

C. Traverse the list and store node values in an array; if a value repeats, a cycle exists.

D. Use two pointers, one moving one step at a time and the other moving two steps at a time; if they meet, a cycle exists. (correct · your answer)

Explanation

The two-pointer technique, also known as Floyd’s Cycle-Finding Algorithm, uses a slow pointer (one step) and a fast pointer (two steps). If the list has a cycle, the fast pointer will eventually meet the slow pointer. This approach uses O(1) space and O(n) time without modifying the list, making it efficient.

Q18 of 20 · Correct · Your answer: B · Easy · Git & Development Workflow

When working with Git, which command should you use to create a new branch and switch to it immediately?

A. git switch

B. git checkout -b (correct · your answer)

C. git merge

D. git branch

Explanation

The command 'git checkout -b ' creates a new branch and switches to it immediately, combining both actions in one step. 'git branch ' only creates the branch but does not switch to it. 'git switch ' switches to an existing branch, and 'git merge ' merges another branch into the current one.

Q19 of 20 · Correct · Your answer: B · Easy · Async Systems & Backend Basics

In an asynchronous backend system, which of the following best describes how the system handles multiple requests efficiently?

A. By duplicating the server hardware to handle more requests concurrently

B. By initiating requests and continuing other work without waiting for the previous requests to finish, using callbacks or promises (correct · your answer)

C. By processing each request one at a time, waiting for each to complete before starting the next

D. By starting all requests simultaneously and blocking until all are completed

Explanation

Asynchronous systems initiate requests and continue processing other tasks without waiting for the previous requests to complete. This is often managed using callbacks, promises, or async/await, allowing efficient handling of multiple requests without blocking.

Q20 of 20 · Correct · Your answer: C · Easy · Deployment & Docker

Which of the following best describes a Docker container?

A. A virtual machine that simulates an entire operating system on a host machine

B. A software development methodology for automating testing

C. A lightweight, standalone, and executable package that includes everything needed to run a piece of software (correct · your answer)

D. A graphical user interface for managing cloud deployments

Explanation

A Docker container packages an application and its dependencies into a single, lightweight, and portable unit that can run consistently across different environments without the overhead of a full virtual machine.

Coding problem responses (3)

Sum of Even Numbers in a List

Problem 1 of 3

6/6 tests passed · all passed

Difficulty: Easy

Problem: Given a list of integers, your task is to calculate the sum of all even numbers present in the list. If there are no even numbers, output 0 . Sample Explanation: For the input list [1, 2, 3, 4, 5] , the even numbers are 2 and 4 . Their sum is 6 .

Input: The input consists of: n : An integer representing the number of elements in the list (0 <= n <= 10 5). arr : A list of n integers separated by spaces.

Output: Output a single integer: the sum of all even numbers in the list.

Constraints: 0 <= n <= 10 5 -10 9 <= arr[i] <= 10 9

Sample input: 5 1 2 3 4 5

Sample output: 6

```
def solve(n, arr):
    """Only edit this function."""
    # TODO: implement
    s=0
    for i in arr:
        if i%2==0:
            s+=i
    return s
    raise NotImplementedError()

if __name__ == "__main__":
    n = int(input())
    arr = []
    if n > 0:
        arr = list(map(int, input().split()))
    result = solve(n, arr)
    print(result)
```

Process REST API User Data and Calculate Average Age

Problem 2 of 3

5/5 tests passed · all passed

Difficulty: Medium

Problem: You are given JSON-formatted user data obtained from a REST API. The data consists of multiple user records, each with a name and age field. Your task is to process this data and compute the average age of all users whose ages are within a given inclusive range [min_age, max_age] . The input is provided as a single JSON string (potentially spanning multiple lines) representing a list of user objects. You must parse this JSON, filter users whose age is between min_age and max_age (inclusive), and output the average age of these users as an integer (rounded down). If no users match the criteria, output 0 . You may assume the JSON is always valid and properly formatted. Sample Explanation: Suppose the input JSON is: [{"name": "Alice", "age": 28}, {"name": "Bob", "age": 35}] and min_age = 25 , max_age = 30 . Only Alice's age (28) falls within the range. So, the output should be 28 (since the average of a single value is itself).

Input: The input consists of: A single integer min_age (1 <= min_age <= 150). A single integer max_age (min_age <= max_age <= 150). A single JSON string (may span multiple lines) representing a list of user records, where each record is a JSON object with string name and integer age fields.

Output: Output a single integer, the floor of the average age of users whose age is within the range [min_age, max_age] . Output 0 if no such users exist.

Constraints: 1 <= min_age <= max_age <= 150 0 <= Number of users <= 10 5 Each user: name is a non-empty string with at most 100 characters, age is an integer (1 <= age <= 150) Input JSON is valid and does not contain extra fields.

Sample input: 25 30 [{"name": "Alice", "age": 28}, {"name": "Bob", "age": 35}]

Sample output: 28

```
import sys
import json

def solve(min_age, max_age, users_json):
    """Only edit this function."""
    # TODO: implemen
    users=json.loads(users_json)
    matching_ages=[
        user["age"] for user in users
        if min_age<=user["age"]<=max_age
    ]
    if not matching_ages:
        return 0
    return int(sum(matching_ages)/len(matching_ages))
    raise NotImplementedError()

if __name__ == "__main__":
    min_age = int(sys.stdin.readline())
    max_age = int(sys.stdin.readline())
    users_json = ''
    for line in sys.stdin:
        users_json += line.strip()
    result = solve(min_age, max_age, users_json)
    print(result)
```

Top Customers by Monthly Purchase Amount

Problem 3 of 3

3/5 tests passed

Difficulty: Hard

Problem: Given a database of customer purchases, you must determine the top customer(s) by total purchase amount for each month. You are given: A list of purchases where each record consists of customer_id , amount , and date (formatted as YYYY-MM-DD). Your task is to, for each month present in the data, output the customer_id and total_amount for the customer(s) with the highest total purchase amount in that month. If there are multiple customers tied for the highest amount in a month, output all of them (one per line, sorted by customer_id ascending). Sample Explanation: Suppose the purchases are: 3 1 100 2023-03-15 2 150 2023-03-16 1 50 2023-03-18 For March 2023, customer 2 has a total of 150, customer 1 has a total of 150. Both are top customers for that month. The output should be: 2023-03 1 150 2023-03 2 150

Input: The first line contains an integer n (number of purchases). Each of the next n lines contains: customer_id (positive integer), amount (integer ≥ 0), and date (YYYY-MM-DD).

Output: For each month (in chronological order), output one line per top customer: YYYY-MM customer_id total_amount If multiple customers tie, output them sorted by customer_id ascending. Months with no purchases should not appear in the output.

Constraints: 1 ≤ n ≤ 10 5 1 ≤ customer_id ≤ 10 6 0 ≤ amount ≤ 10 9 Dates are valid and range from 2000-01-01 to 2099-12-31

Sample input: 3 1 100 2023-03-15 2 150 2023-03-16 1 50 2023-03-18

Sample output: 2023-03 1 150 2023-03 2 150

```
def solve(n, purchases):
    """Only edit this function."""
    # purchases is a list of tuples: (customer_id, amount, date)
    # TODO: implement
    d={}
    for c,a,dt in purchases:
        m=dt[:7]
        if m not in d:
            d[m]={}
        d[m][c]=d[m].get(c,0)+a
    ans=[]
    for m in sorted(d):
        mx=max(d[m].values())
        for c in sorted(d[m]):
            if d[m][c]==mx:
                ans.append(f"{m} {c} {mx}")
    return ans
    raise NotImplementedError()

if __name__ == "__main__":
    n = int(input())
    purchases = []
    for _ in range(n):
        parts = input().strip().split()
        customer_id = int(parts[0])
        amount = int(parts[1])
        date = parts[2]
        purchases.append((customer_id, amount, date))
    result = solve(n, purchases)
    if result:
        for line in result:
            print(line)
```